SAMPLE TABLES AND DATA

```
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    name VARCHAR(50),
    dept_id INT,
    salary INT,
    experience INT
);

INSERT INTO employees (emp_id, name, dept_id, salary, experience) VALUES
(1, 'Alice', 101, 50000, 2),
(2, 'Bob', 101, 70000, 5),
(3, 'Charlie', 102, 60000, 4),
(4, 'David', 102, 60000, 3),
(5, 'Eve', 103, 90000, 8),
(6, 'Frank', 103, 40000, 1),
(7, 'Grace', 101, 70000, 6),
(8, 'Helen', 104, NULL, 2);
```

⚡ ROUND 1: Basic Aggregate Questions

Try writing SQL for these:

Q1

Find the **total number of employees**.

```
select count(emp_id) from employees;
```

Q2

Find the **average salary** of all employees.

```
select avg(salary) from employees;
```

`AVG()` **automatically ignores NULL**

Q3

Find the **maximum salary** in the company.

select max(salary) from employees;

Q4

Find the **minimum experience**.

select min(experience) from employees;

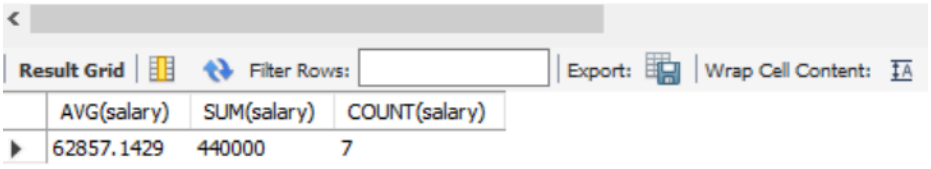
Q5

Find the **number of employees who have salary NOT NULL**.

select count(salary) from employees;
`COUNT(salary)` counts **only NON-NULL salaries** unlike COUNT(*) which counts all rows.

Clarity Query-

```
1    SELECT AVG(salary), SUM(salary), COUNT(salary)
2    FROM employees;
```



💡 Why does this work without GROUP BY?

SELECT AVG(salary), SUM(salary), COUNT(salary)
FROM employees;

✅ Key Idea:

When you use **aggregate functions ONLY**, SQL treats the **entire table as one group**.

👉 Think of it like:

> "Group everything together into one bucket, then calculate aggregates."

🧠 Mental Model

Your table:

emp_id	salary
...	...

SQL internally does:

GROUP ALL ROWS → one single group

Then computes:

- AVG(salary)
- SUM(salary)
- COUNT(salary)

⚠️ When do you NEED `GROUP BY`?

You need `GROUP BY` when:

👉 You mix **aggregate + non-aggregate columns**

❌ This will FAIL:

```
SELECT dept_id, AVG(salary)
FROM employees;
```

💥 Error:

> "dept_id must appear in GROUP BY"

✅ Correct version:

```
SELECT dept_id, AVG(salary)
FROM employees
GROUP BY dept_id;
```

🔥 Interview Rule (VERY IMPORTANT)

👉 Memorize this:

> If SELECT contains a column that is NOT aggregated → it MUST be in GROUP BY

This will also work because only aggregates there-

```
1    SELECT AVG(experience), SUM(salary), COUNT(emp_id)
2    FROM employees;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

AVG(experience)	SUM(salary)	COUNT(emp_id)
3.8750	440000	8

⚡ ROUND 2: GROUP BY

Q6

Find **number of employees in each department**.

SELECT COUNT(*), dept_id

FROM employees

GROUP BY dept_id;

This works too-

```
1 •  SELECT COUNT(*)
2    FROM employees
3    GROUP BY dept_id;
```

Result Grid | Filter Rows:

COUNT(*)
3
2
2
1

Q7

Find **average salary per department**.

SELECT AVG(salary), dept_id

FROM employees

GROUP BY dept_id;

`AVG()` ignores NULL → dept 104 will return **NULL**

```
1 •  SELECT AVG(salary), dept_id
2    FROM employees
3    GROUP BY dept_id;
```

Result Grid | Filter Rows: | Export:

AVG(salary)	dept_id
63333.3333	101
60000.0000	102
65000.0000	103
NULL	104

Q8

Find **maximum salary in each department**.

SELECT MAX(salary), dept_id

FROM employees

GROUP BY dept_id;

Q9

Find **total salary paid per department**.

SELECT SUM(salary), dept_id
FROM employees
GROUP BY dept_id;

NULL salaries are ignored in SUM

```
1 •  SELECT SUM(salary), dept_id
2    FROM employees
3    GROUP BY dept_id;
```

Result Grid | Filter Rows: | Export:

SUM(salary)	dept_id
190000	101
120000	102
130000	103
NULL	104

🧠 Scenario

Department 104:

emp_id	dept_id	salary
8	104	NULL

So **all salary values are NULL**

✅ What each aggregate returns

🔢 `COUNT`

- `COUNT(*)` → **1** (counts rows)
- `COUNT(salary)` → **0** (counts non-null values)

✔️ You already got this right

💰 Now the important part:

👉 `SUM(salary)`

➡️ **Returns NULL**, NOT 0

👉 `AVG(salary)`

➡️ **Returns NULL**, NOT 0

👉 `MIN(salary)`

➡️ **Returns NULL**

👉 `MAX(salary)`

➡️ **Returns NULL**

❗ WHY not 0?

Because:

SQL treats NULL as “unknown”, not as 0

So:

- No valid values exist
- SQL cannot compute → returns **NULL**

🧠 Mental Model

Think:

SUM of nothing = NULL
AVG of nothing = NULL
MIN/MAX of nothing = NULL

NOT 0 ❌

⚠️ SUPER IMPORTANT INTERVIEW FOLLOW-UP

👉 How to convert NULL → 0?

✅ Use COALESCE

```
SELECT dept_id, COALESCE(SUM(salary), 0) AS total_salary
FROM employees
GROUP BY dept_id;
```

🚀 Key Takeaway (Memorize This)

Function	All NULL values → result
COUNT(*)	Number of rows
COUNT(column)	0
SUM	NULL
AVG	NULL
MIN / MAX	NULL

⚡ ROUND 3: HAVING (INTERVIEW FAVORITE ⚠️)

Q10

Find departments having **more than 2 employees**.

SELECT dept_id, COUNT(emp_id)

FROM employees

GROUP BY dept_id

HAVING COUNT(emp_id) > 2;

Q11

Find departments where **average salary > 60000**.

SELECT dept_id, AVG(salary)

FROM employees

GROUP BY dept_id

HAVING AVG(salary) > 60000;

Q12

Find departments where **total salary > 150000**.

SELECT dept_id, SUM(salary)
FROM employees
GROUP BY dept_id
HAVING SUM(salary) > 150000;

✅ 1. Prefer `COUNT(*)` over `COUNT(emp_id)`

```sql
HAVING COUNT(*) > 2
```

👉 Reason:

- More generic
- Works even if column changes / nullable

✅ 2. Add aliases (clean + professional)

```sql
SELECT dept_id, COUNT(*) AS emp_count
FROM employees
GROUP BY dept_id
HAVING COUNT(*) > 2;
```

⚠️ CRITICAL INTERVIEW CONCEPT

👉 WHERE vs HAVING (you *must* nail this)

Clause	Works on	When applied
WHERE	Rows	BEFORE grouping
HAVING	Groups	AFTER grouping

❌ This is WRONG:

```sql
WHERE COUNT(*) > 2
```

👉 Aggregates not allowed in WHERE

❓ What does this return?

```sql
SELECT dept_id, AVG(salary)
FROM employees
WHERE salary > 50000
GROUP BY dept_id;
```

VS

```sql
SELECT dept_id, AVG(salary)
FROM employees
GROUP BY dept_id
HAVING AVG(salary) > 50000;
```

👉 These are **NOT the same**

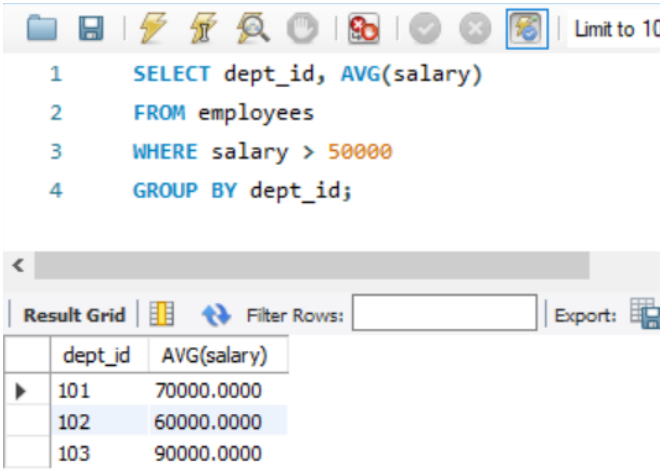
Clarity-

```sql
select * from employees;
```

Result Grid | Filter Rows: | Edit: | Export

emp_id	name	dept_id	salary	experience
1	Alice	101	50000	2
2	Bob	101	70000	5
3	Charlie	102	60000	4
4	David	102	60000	3
5	Eve	103	90000	8
6	Frank	103	40000	1
7	Grace	101	70000	6
8	Helen	104	NULL	2
NULL	NULL	NULL	NULL	NULL

1st query returns average salary calculated by using only salaries greater than 50000 for each department-



✔️ **What it does:**

Filters rows **before grouping**, then calculates average.

👉 So:

- Only employees with `salary > 50000` are considered
- Then grouped by `dept_id`
- Then AVG is calculated

📌 **Key idea:**

Some employees are completely excluded from calculation

🔥 Interview-Level Difference

Aspect	Query 1 (WHERE)	Query 2 (HAVING)
Filtering stage	Before aggregation	After aggregation
Rows used in AVG	Only salary > 50000	All salaries
Effect	Changes the average	Filters based on average

⚡ ROUND 4: TRICKY / EDGE CASES (VERY IMPORTANT)

Q13

What will `COUNT(salary)` return vs `COUNT(*)`?

`COUNT(salary)` excludes NULL, `COUNT(*)` includes all rows

Q14

Find number of employees per department **including departments with NULL salary values correctly counted**.

SELECT dept_id, COUNT(*) AS emp_count

FROM employees

GROUP BY dept_id;

Q15

Find departments where **all employees have salary > 50000**.

SELECT dept_id

FROM employees

GROUP BY dept_id

HAVING MIN(salary) > 50000;

FOR Q14-

Ahh got it — you're asking:

> "If I want **dept-wise aggregates BUT also show salary**, how do I do it without breaking GROUP BY?"

This is a **very common confusion** — let's clear it properly 🔥

🚨 Core Rule (You MUST remember)

If you use `GROUP BY`, every column in `SELECT` must be:

- either in `GROUP BY`
- or inside an aggregate function

This will show unique salaries in each department-

```sql
select * from employees;

SELECT COUNT(*), dept_id, salary
FROM employees
GROUP BY dept_id, salary;
```

Result Grid | Filter Rows: | Edit:

emp_id	name	dept_id	salary	experience
1	Alice	101	50000	2
2	Bob	101	70000	5
3	Charlie	102	60000	4
4	David	102	60000	3
5	Eve	103	90000	8
6	Frank	103	40000	1
7	Grace	101	70000	6
8	Helen	104	NULL	2

employees 6 | Result 7

Result Grid | Filter Rows:

COUNT(*)	dept_id	salary
1	101	50000
2	101	70000
2	102	60000
1	103	90000
1	103	40000
1	104	NULL

Another variation-

```sql
SELECT COUNT(*), dept_id, salary
FROM employees
GROUP BY dept_id, salary, name;
```

Result Grid | Filter Rows:

COUNT(*)	dept_id	salary
1	101	50000
1	101	70000
1	102	60000
1	102	60000
1	103	90000
1	103	40000
1	101	70000
1	104	NULL

🧠 So what are your options?

✅ Option 1: Show aggregated salary (most common)

If your goal is **per department summary**, then:

```
SELECT dept_id,
    COUNT(*) AS emp_count,
    AVG(salary) AS avg_salary,
    SUM(salary) AS total_salary
FROM employees
GROUP BY dept_id;
```

👉 You don't show raw salary, only **aggregated version**

✅ Option 2: Include salary → then you MUST group by it

```
SELECT dept_id, salary, COUNT(*)
FROM employees
GROUP BY dept_id, salary;
```

👉 But this changes meaning:

- Now it's **count per (dept, salary)**
  NOT per department

✅ Option 3: Use window functions (🔥 BEST for interviews)

If you want:

> "Show each employee's salary AND dept-level aggregates"

```
SELECT emp_id, dept_id, salary,
    COUNT(*) OVER (PARTITION BY dept_id) AS dept_count,
    AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg
FROM employees;
```

💡 This gives:

emp_id	dept_id	salary	dept_count	dept_avg

👉 No grouping collapse
👉 Row-level + aggregate together

🧠 Interview Insight

When interviewer asks:

> “Can you show both row data and aggregate?”

👉 They expect:

- ❌ NOT GROUP BY
- ✅ WINDOW FUNCTIONS

🔥 Quick Decision Guide

Goal	Use
One row per dept	GROUP BY
Keep all rows + aggregates	WINDOW FUNCTIONS
Breakdown by salary	GROUP BY dept_id, salary

⚡ ROUND 5: REAL INTERVIEW LEVEL

Q16

Find the **second highest salary** (without window functions).

SELECT MAX(salary)

FROM employees

WHERE salary < (SELECT MAX(salary) FROM employees);

Q17

Find departments where **max salary > overall average salary**.

SELECT dept_id

FROM employees

GROUP BY dept_id

HAVING MAX(salary) > (SELECT AVG(salary) FROM employees);

💡 Explanation:

- `GROUP BY dept_id` → per department
- `MAX(salary)` → dept-wise max
- Compare with overall avg

This query is wrong as it selects just dept_id of department where salary is same as max salary in entire table and is greater than average of entire table salaries-

❌ This part (for Q17) needs correction

```sql
SELECT dept_id
FROM employees
WHERE salary = (SELECT MAX(salary) FROM employees)
AND salary > (SELECT AVG(salary) FROM employees);
```

❗ Problem:

- You are comparing **global max salary**, not **dept-wise max salary**
- Question asks:

> "max salary of EACH department > overall avg salary"

Q18

Find department with **highest total salary payout**.

```
SELECT dept_id

FROM employees

GROUP BY dept_id

HAVING SUM(salary) = (

   SELECT MAX(total_salary)

   FROM (

      SELECT SUM(salary) AS total_salary

      FROM employees

      GROUP BY dept_id

   ) t

);
```

💡 Logic:

1. Inner query → total salary per dept
2. Find max of those
3. Outer query → return dept with that value

Second method-

```
SELECT dept_id

FROM employees

GROUP BY dept_id

ORDER BY SUM(salary) DESC

LIMIT 1;
```

🧠 Interview Tips (VERY IMPORTANT)

🔹 For “Nth highest”

Use:

MAX(...) WHERE < previous max

🔹 For “per group condition”

Use:

GROUP BY + HAVING

🔹 For “top group”

Use either:

- ORDER BY + LIMIT ✅
- OR subquery (safer for ties)

😈 Final Challenge (Last Stretch)

Try this:

👉 Find **employees who earn more than their department average**

Wrong query (use alias for outer query table not inner query table)-

select * from employees

where salary > (select avg(e.salary)

from employees e where dept_id = e.dept_id);

Great question — and this is a **very important distinction**.
It's **NOT just about aliases** — it's about **correlation** (linking inner and outer queries).

🔍 Your Query vs Correct Query

❌ Your Query

```
SELECT *

FROM employees

WHERE salary > (

   SELECT AVG(e.salary)

   FROM employees e

   WHERE dept_id = e.dept_id

);
```

✅ Correct Query

```
SELECT *

FROM employees e1

WHERE salary > (

   SELECT AVG(e2.salary)

   FROM employees e2

   WHERE e2.dept_id = e1.dept_id

);
```

🚨 Core Difference (THIS is the real issue)

❌ Your condition:

dept_id = e.dept_id

👉 This compares:

- `dept_id` (from subquery table)
- `e.dept_id` (same subquery table)

So it becomes:

e.dept_id = e.dept_id → ALWAYS TRUE

👉 Meaning:

- No filtering happens
- Subquery becomes:

SELECT AVG(salary) FROM employees

➡️ **Global average**, NOT dept-wise ❌

✅ Correct Query Logic

e2.dept_id = e1.dept_id

👉 This compares:

- `e2.dept_id` → inner query
- `e1.dept_id` → outer query

👉 Now SQL does:

> “For THIS employee (outer row), compute avg of THEIR department”

✔️ This is called a **correlated subquery**

🧠 So is alias the issue?

👉 ❌ **No — alias is not the root problem**
👉 ✅ **Missing correlation is the real problem**

Aliases just help you **refer correctly across scopes**

🔥 Important Concept: Query Scope

Think of it like this:

Query	Scope
Outer query	`employees` (current row)
Inner query	`employees e` (separate table instance)

👉 Without aliasing:

- SQL cannot distinguish inner vs outer clearly
- You accidentally compare inner with itself

🧠 Mental Model (VERY IMPORTANT)

Your query:

AVG(salary of ALL employees)

Correct query:

AVG(salary of employees WHERE dept = current employee's dept)

🔥 Interview Gold Statement

> “A correlated subquery must reference the outer query to filter per-row context. Without that, it becomes an uncorrelated subquery.”

⚡ Bonus: Can your query be fixed without alias?

Technically yes, but messy and unclear:

```
SELECT *
FROM employees e1
WHERE salary > (
   SELECT AVG(salary)
   FROM employees
   WHERE employees.dept_id = e1.dept_id
);
```

👉 Still uses alias for outer query → unavoidable for clarity
